1. Sommaire Exécutif & Introduction Générale

Ce projet d'ingénierie présente la conception, le provisionnement automatisé et la validation d'une **plateforme Cloud sécurisée Kubernetes sur AWS** intégrant une stack **DevSecOps et d'Incident Response (IR) automatisée**.

L'objectif principal est d'apporter une réponse immédiate aux cyberattaques ciblées (Log4Shell, SQL Injection, XSS, SSRF) en réduisant le temps moyen de réponse (MTTR) à **moins de 2 secondes**. L'infrastructure est entièrement provisionnée via **Terraform** (Infrastructure as Code) et héberge une application e-commerce (PHP/Symfony) protégée par un pipeline de détection hybride (Machine Learning) et un orchestrateur de playbooks d'atténuation basé sur la méthodologie **SANS PICERL**.

2. Contexte et État de l'Art

Dans les environnements Cloud Native, la conteneurisation accélère le déploiement mais élargit la surface d'attaque. La gestion des incidents s'articule autour des 6 phases standard de la norme SANS PICERL : *Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned*. L'introduction du Machine Learning permet de scorer dynamiquement la gravité des requêtes avant d'isoler l'attaquant au niveau réseau et conteneur.

3. Analyse des Besoins

- **BF-1 (Ingestion & Parsing IP) :** Ingestion temps réel des logs Ingress NGINX et extraction de l'IP source réelle via X-Forwarded-For.
- **BF-2 (Scoring ML Hybride) :** Évaluation asynchrone du niveau de menace (0 à 100) combinant signatures OWASP et scoring comportemental.
- **BF-3 (Orchestration IR PICERL) :** Déclenchement automatique du playbook full_advanced_protection pour les scores >= 76 ou attaques critiques.
- **BF-4 (Mesures d'Atténuation) :** Blocage IP progressif Redis, rate limiting adaptatif glissant, quarantaine logique de pod, génération de NetworkPolicies K8s, capture forensic et redirection Honeypot.
- **BF-5 (Supervision & Alertes) :** Interface Next.js avec bouton synchrone EXECUTE NOW et Webhooks HTTP JSON vers Slack et SIEM.

4. Conception de l'Architecture AWS et Kubernetes

4.1. Topology AWS & Instance Specification

Instance EC2	Type / Config	Subnet	Rôle & Service Exécuté
k8s-bastion	t3.micro (1 vCPU, 1GB)	Public A	Point d'accès d'administration SSH
k8s-master	t3.medium (2 vCPU, 4GB)	Privé A	Control Plane K8s (API Server, etcd)
k8s-worker-1	t3.medium (2 vCPU, 4GB)	Privé A	Worker 1 : Symfony, FastAPI, Redis, Postgres
k8s-worker-2	t3.medium (2 vCPU, 4GB)	Privé B	Worker 2 : Service ML, Celery, Frontend

4.2. Flux Réseau Utilisateur & Limitation Technique

Flux de Trafic : Internet → AWS ALB → NodePort 32080 → NGINX Ingress → Service Kubernetes → Pod Applicatif.
Précision Architectural : Le CNI Flannel (Pod CIDR 10.244.0.0/16) assure la connectivité Pod-to-Pod mais n'applique pas seul les NetworkPolicies au niveau du noyau. Les manifests YAML sont donc générés et stockés dans Redis (l'implémentation de Calico/Cilium constitue une perspective). De même, l'architecture comporte un Control Plane unique.

5. Implémentation avec Terraform (IaC)

Fichier Terraform	Description et Ressources Provisionnées
provider.tf	Spécification du provider AWS et de la région us-east-1.
networking.tf	Création du VPC, Subnets publics/privés, IGW, NAT Gateway, EIP et tables de routage.
security_groups.tf	Définition des Security Groups (Bastion, Master, Workers, ALB).
compute.tf	Provisionnement des 4 instances EC2 et attachement des userdata et clés SSH.
loadbalancer.tf	Configuration de l'ALB AWS, Target Group (NodePort 32080) et Listener HTTP (80).
flowlogs.tf / guardduty.tf	Activation des VPC Flow Logs vers CloudWatch Logs et activation d'AWS GuardDuty.
userdata-*.sh	Scripts Shell de bootstraping installant containerd, Kubeadm, Flannel et NGINX Ingress.

6. Déploiement de l'Application & Stack SecOps

Attaque / Vulnérabilité	Endpoint / Vecteur d'Injection	Sévérité ML & Impact
Log4Shell / Cmd Injection	Header HTTP User-Agent: ${jndi:ldap://...}	**CRITICAL (90+)** — Exécution de code à distance
SQL Injection (SQLi)	Paramètre /search?q=' OR 1=1--	**CRITICAL (85+)** — Exfiltration de données
Cross-Site Scripting (XSS)	Avis produit <script>steal()</script>	**HIGH (65+)** — Vol de session utilisateur
SSRF	Import /api/preview?url=http://169.254.169.254	**HIGH (60+)** — Reconstitution d'accès IAM
Brute Force Login	Requêtes répétées sur /login	**MEDIUM (50+)** — Auth / Déni de service

7. Détection et Scoring du Risque (Service ML)

Le service ML (FastAPI/Scikit-Learn, port 8001) repose sur un **moteur hybride de détection et de scoring** associant la recherche de signatures OWASP et un modèle statistique comportemental.
Formule du score composite : Score = 0.50 * Security + 0.30 * Reliability + 0.20 * Frequency.

8. Automatisation de l'Incident Response (PICERL)

Phase PICERL	Action du Playbook	Niveau d'Implémentation
CONTAINMENT	Blocage IP Progressif	Écriture Redis : 5m, 30m, Blacklist 24h.
CONTAINMENT	NetworkPolicy K8s	Génération du manifest YAML stocké dans Redis.
CONTAINMENT	Quarantaine du Pod	Label quarantine=active et Taint enregistré dans Redis.
ERADICATION	Rate Limit Adaptatif	Fenêtre glissante Redis avec backoff exponentiel.
ERADICATION	Redirection Honeypot	Règle de dérivation vers Pod piège enregistrée dans Redis.
LESSONS LEARNED	Forensic Capture	Archivage JSON/Redis des logs, manifests, pcap (30j).

9. Intégration et Pipeline de Traitement à 13 Étapes

```
1. Événement généré -> 2. Ingest JSON POST /logs/ingest -> 3. Parsing IP source -> 4. Stockage
PostgreSQL raw_logs -> 5. Evaluation ML -> 6. Score calculé -> 7. Incident créé si score > 40 -> 8.
Dispatch Celery si score >= 76 -> 9. Exécution playbook par Celery -> 10. Inscription état de
```

```
sécurité dans Redis -> 11. Enregistrement action dans PostgreSQL -> 12. Mise à jour Dashboard
Next.js -> 13. Émission Webhooks Slack & SIEM.
```

10. Validation Expérimentale & Démonstration

La validation sur l'Incident #184 (Attaque Log4Shell lancée depuis 32.197.70.124) confirme le peuplement synchrone de Redis :

```
$ docker exec k8s_redis redis-cli keys "ir:*" 1) "ir:ratelimit:32.197.70.124:ingress-nginx" 2)
"ir:forensic:184-20260520200757" 3)
"ir:quarantine:ingress-nginx:ingress-nginx-controller-5458dd5f6-v26dd" 4)
"ir:blocked_ip:32.197.70.124" 5)
"ir:networkpolicy:ingress-nginx:ingress-nginx-controller-5458dd5f6-v26dd" 6)
"ir:honeypot:redirect:32.197.70.124"
```

11. Limites et Perspectives d'Évolution

- **CNI Calico / Cilium :** Remplacement de Flannel pour appliquer réellement les NetworkPolicies K8s au niveau eBPF/IPtables.
- **Falco (eBPF) :** Détection d'anomalies au niveau des appels système du noyau Linux sur les Workers EC2.
- **HA Control Plane :** Évolution vers un Control Plane multi-masters hautement disponible.
- **Gouvernance & GitOps :** Intégration de Kyverno (admission control), Trivy (image scanning) et ArgoCD (GitOps).

12. Conclusion Générale

Ce projet d'ingénierie à TEK-UP démontre la faisabilité et l'efficacité d'un pipeline DevSecOps / Incident Response automatisé. En associant l'Infrastructure as Code (Terraform), Kubernetes, le Machine Learning et la méthode SANS PICERL, la plateforme réduit le temps de réaction à moins de 2 secondes tout en assurant une auditabilité complète.